Introduction to JavaScript

Console operations

```
console.log("Anything")
```

alert

```
alert("Any txt")
```

prompt

```
var a = prompt("Enter the no.")
```

Confirm

```
var a = confirm("Any yes/No type question").
```

Changing css properties and tags using JS

```
document.title = "Any txt"
document.body.style.background-color = "red"
```

Element — Property (Attribute) — Property — Value

JS Variables

var variable_name = value.

Ex:-

~~Var~~
var a = 5;
var b = "Atharva";

typeof a. → returns the data type of any variable

Let & var.

~~Var is Scope independent.~~
~~let is Scope dependent.~~

→ Var is globally Scoped.
→ let is block Scoped.

type Conversions. (Explecit)

Number ("123") // 123
Number ("abc") // NaN

parseInt ("12.5") //12
parseFloat ("12.5") //12.5

Java Script Object

```
let o = {
        "name" : "Atharva",
        "Job" : "Programmer",
        "Id" : 1008
      }

o.salary = "100 crores"
```

→ Appending ^ any key value pair or modifying.

JS Conditionals

Syntax:-

```
if (expression) {
    // Code }
else {
    // Code if the 'if' expression is false.
}

if (exp) {
    // code.
} else if (exp2) {
    // code 1
} else {
    // this will invoke if all the above exp are false
}
```

Note.

* == Compares only value not type.

"3" == 3 // this is true in JS

* use === to compare value & type both.

"3" === 3 // this will return false.

3 === 3 // this will return true.

Conditional Statements.

(exp1) ? (exp2) : (exp3)

if exp1 returns true, exp2 will be evaluated and if exp1 returns false, exp3 will be evaluated.

Loops in Js

for loop

```
for (let i=0; i<10; i++){
    // code.
}
```

for-in loop. (used to itrate in any object.)

→ this is any object.

```
for (const key in obj) {
        const element = obj[key];
        console.log(key, element)
}
```

for-of loop. (used to itrate in strings)

```
for (const c of "Atharva"){
    console.log(c)
}
```

While ~~Condition~~ loop

```
while (expression){
    // code.
}
```

do-while loop.

```
do {
    // code.
} while (expression);
```

Functions in JS

Syntax:

```
function <function_name>(<Parameters>) {
        //Code.
}
```

Example:

```
function Hello(name) {
        console.log("Hey" + name);
}
```

Arrow functions

```
const funcs = (n) => {
        console.log("I am an arrow fxn", n)
};
```

Strings (Immutable)

```
let a = "Harry";
```

Accessing the Strings

a[index]

a[0], a[1], etc.

→ a.length returns length of the array.

template literals

let name1 = "Atharva"
let friend = "Abc"
str = `His name is ${name1} and his friends name is ${friend}`

backtic

Escape Sequence.

\n → Newline. \r → Carriage Return
\t → tab

Methods.

.toUpperCase() // to make string Capital.

.toLowerCase() // to make string Small.

.split("anything") // splits the string to array on "anything" where "anything" is omitted.

Slicing.

.slice(start_index, end_index)

replacing.

.replace("sh", "77")

replaces 1st occourence with the specified.
sh will be replaced by 77

.trim() // removes whitespaces from Strings.

~~.idex~~

.indexOf ("any value") // returns the index of 1st occourence of char or substring of any string.

Array (mutable)

Syntex :- typeof (arr) => object

let arr = [1, 2, 3, 4, 5]
let arr1 = [1, 2, "Atharva", 3]

accessing array items.

arr[index]

Example

arr[0]
=> 1

changing array Element Values

arr[0] = 2.

Array Methods

Object.values (any_obj) returns itrable containing values

Object.Keys (any_obj) returns itrable containing Keys.

→ map() (creates a new array by performing some operations on each array element)

arr.map((value, inden, array) => { return value * value });

Array Methods.

→ .toString() // Converts array to comma. Seperated string.

→ .join("anything") // converts array to "anything" seperated string.

→ .pop() // returns and removes the last element of the array.

→ ~~.sort~~ .sort() // sorts the array.

~~.push()~~

→ .push(item) // appends the item in array & returns the updated length

→ .shift() // returns & removes the First element of the array.

→ .unshift(item) // add the item @ start in array & returns the updated length.

→ # delete Keyword.

delete arr[inden]

deleats the value @ inden but space remains allocated

Array.

slicing.

→ .splice (start , end)

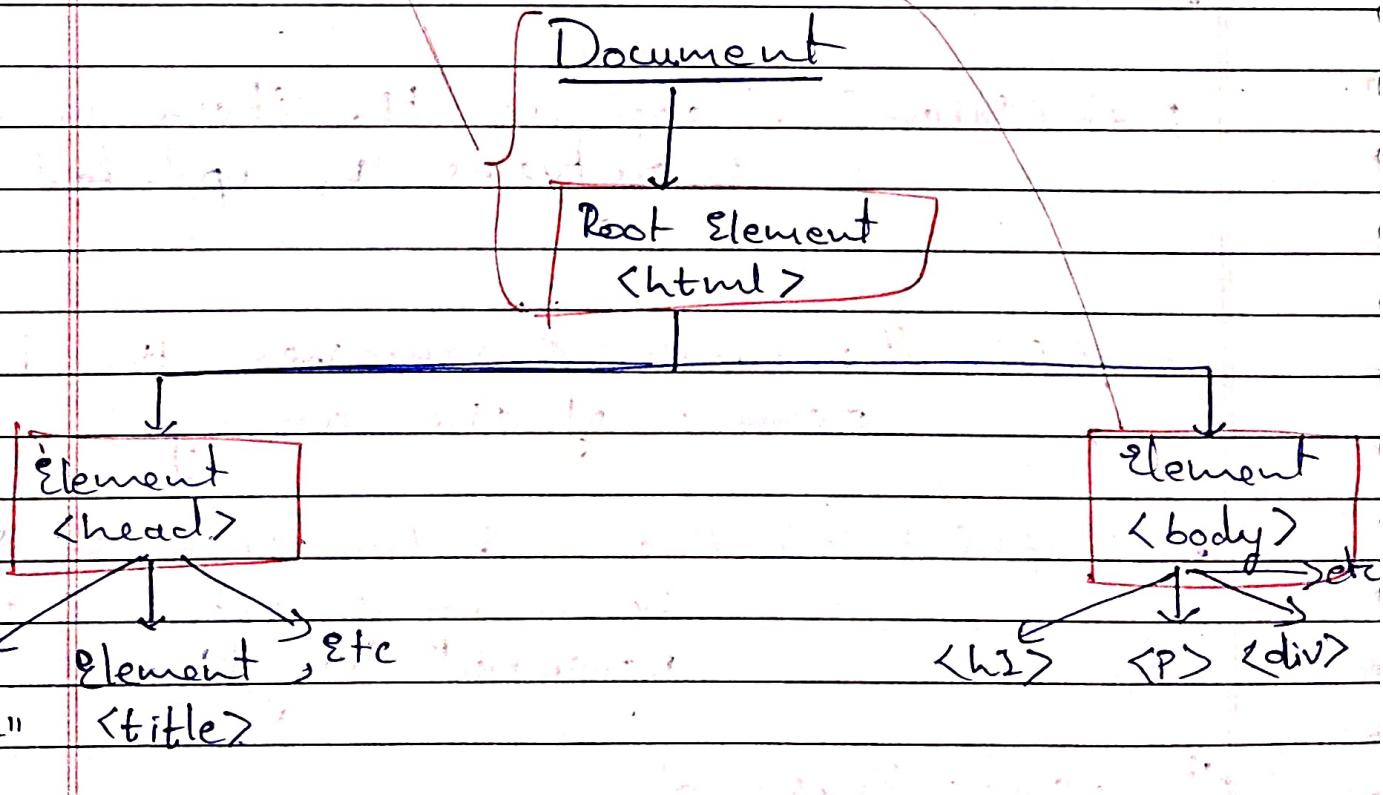
Indices

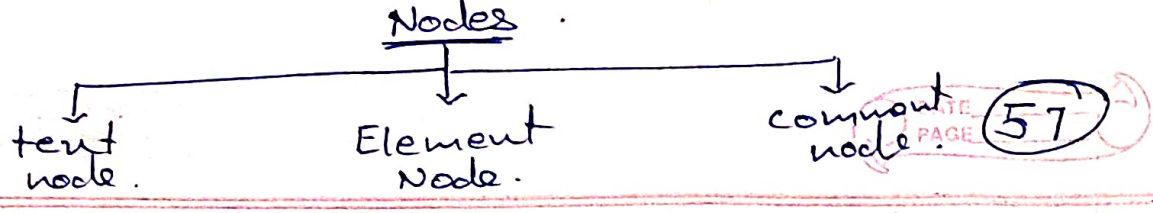
Document Object Model

→ Used to target HTML Elements in JS

document.body.style.backgroundColor = "red"

- document → object for target big DOM
- body → HTML Body (or any Element)
- style → attribute
- backgroundColor → Property & Value.





Children, Parent & Sibling Nodes in DOM JS.

① .childNodes

document.body.childNodes
or
document.body.childNodes[index]

Returns includes all nodes (text, Elements, etc.)

Nesting.

.childNodes[1].childNodes

firstChild, lastChild & childNodes.

- firstChild → first child element
- lastChild → last child element
- &childNode → All Child nodes.

Elements only navigation.

② .children .firstElementChild .lastElementChild
This includes HTML Elements only.

③ Siblings (children of the same parent)

- previousElementSibling
- nextElementSibling.

~~firstElementChild~~
~~lastElementChild~~

Selecting by Ids, Classes & more

by IDs

const a = document.getElementById("any-Id")

a.style or a.value, etc

by Class.

document.getElementsByClassName("any-class")

~~returns HTML list of all the Elements of same class.~~

→ returns HTML collection containing all the elements of same class

CSS selector type addressing (querySelector)

document.querySelector("css selector")

→ returns the 1st matching element.

document.querySelectorAll("css selector")

→ returns HTML Collection containing all matching elements.

by tag name.

document.getElementByTagName("div")

returns an HTML collection containing all "div".

Matches, closest & contains methods.

* element.matches(css) → to check if the element matches the given css selector

* element.closest(css) → to look for the nearest ancestor that matches the given css selector The element itself is also checked.

* elementA.contains(elementB) → Returns true if elementB is inside elementA or elementA == elementB

For Each. (used with arrays.)

Example Syntax:-

```
Array.from(htmlcollection).forEach(
    element => {
        //code using 'element'
    }
)
```

Inserting & Removing HTML Elements Using JS

→ element.innerText
returns text inside any element

→ element.innerHTML
returns HTML inside the element (element itself not included)

→ element.outerHTML
returns HTML inside and also itself the element.

above mentioned can also be used to modify the respective HTML

Example.

```
element.innerHTML = `
<div>
<h1> Hello </h1>
<P> Hi, I am Atharva </P>
</div>
`
```

Attribute Methods

.hasAttribute (name)
Method to check for existing of an attribute.

.getAttribute (name)
returns the value of the specified attribute

.setAttribute (name, value)
sets the attribute & value.

.removeAttribute (name)
removes the specified Attribute.

.attributes
return collection of all attributes.

Class.

.classList
returns list of all classes.

.className
returns string containing all classes.

.classList.add ("any_class")
.classList.remove ("any_class")
} add or remove any class.

.classList.toggle ("any_class")

Events

```
element.addEventListener("anyevent", () => {
    // What to do even the event is occoured.
})
```

Example.

```
button.addEventListener("click, () => {
        // Code.
})
```

Event Bubbling

When any event is occoured on any element it is also automatically occour on is all parents.
This is called Propagation.
to stop this :—

```
ele.addEventListener("Event", (e) => {
    #e
    e.stopPropagation()
    // code.
}
```

set interval.

to do anything infinitely at a specific interval

setInterval (handler , time_in_ms)

Example

```
setInterval ( () => {
    // code.
    }, 3000);
```

to pause it
use :-
clearInterval (same_time_in_ms)

set Timeout

to do anything once at a specific interval.

setTimeout (handler , time_in_ms)
clearTimeout (same_time_in_ms)

Syntax is similar as setInterval.

JS's async nature.

* Asynchronous actions are the actions that we initiate now and they finish later
Example:-
setTimeout

* Synchronous actions are the actions that initiate and finish one-by-one.

Callback function.

A Callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete an action.

Callback doom

We call a callback inside a function and another callback in that callback & so on then this will make an callback doom.

Solution of this is Promises

Promises

Promise of Code execution.

Syntax:-

```
let promise = new Promise ( function (resolve, reject ) {  // Promise constructor.
                                # executor.
                                resolve (any-value) }
);
```

promise.then .then. & .catch.

```
promise.then ((a) => {
        # Code execution after resolving
        # the promise, with 'a' as resolve value
}).catch ((err) => {
        # Code when Promise is rejected.
        # with 'err' as the error value.
})
```

Promise API

← Harry notes → Pg :- 56

Async & Await

→ Special Syntax to work with promises in JS

→ a fxn can be made async by :-

```
async function fxn-name () {
    # Code.
}
```

→ An async fxn always returns a promise other values are wrapped in a promise automatically.

→ we can do something like :-

```
fxn-name.then ( # e => {
        # Code. when fxn-name is resolved.
})
```

→ So, it ensures that the function returns a promise and wraps non promises in it.

await (works only inside async functions)

let value = await promise

It makes the JS code wait until the promise settles & returns its value.

Fetch API

→ ~~its a function~~ which takes URL as input and returns a promise.
→ async / await is used

Syntax

let data = await fetch (url)

here 'data' is a promise object
and can :-

data.then (# code) . catch (# code)

Errors handling.

* throwing custom errors.

Syntax:-

```
throw SyntaxError("Error msg")
```

SyntaxError → Error type
"Error msg" → Error Msg.

* Error handling.

Syntax:-

```
try {
  // Code.
} catch (error) {
  // code when 'error' occours.
}
} finally {
  // code exicuted finally.
}
```

(error) contains an error object

finally

① It will run irrespective of try & catch.
② When used inside function definations and functions ~~are~~ is returned in try and catch block then also finally will run.

try catch works Synchronously.

if error occours in scheduled code, like in setTimeout, then try... catch wont catch it:

```
try {
  setTimeout(fxn() {
        //error code. ⟶ Script dies &.
  }                    catch wont work.
}catch ...
```

That's because the fxn itself is executed later, when the engine has already left the try... catch construct.

error object.

error.name. ⟶ returns name of the error
error.message. ⟶ returns message of error.
error.stack.

OOPs in JS

Classes

```
class Class_Name {

  constructor ( Parameters ) {
    // constructor code.
  }

  any_function() {
    // code.
  }

  any_property = 2

}
```

creating object of class

```
let a = new Class_Name (Parameters)
```

Inheritance

```
class derived_class extends base_class {

  // code.

}
```

Super Keyword.

if a base class has an constructor then to run it in derived class we have to use `super(Parameters)`

```
class base {
    constructor(Parameters) {
        console.log('base')
    }
    // code.
}

class derived extends base {
    constructor(Parameters_1) {
        super(Parameters_1)
        console.log('derived')
    }
    // code
}
```

instance of

any_object instance of any_class.

returns true if 'any_object' is instance of 'any_class' directly or indirectly (inherited from) otherwise returns false.

Advance JS

IIFE (Immediatly Invoked Function Expression)

Syntax :-

formal Parameters

```
(function fun_name () {
        // code.
})()
```

argumants.

→ its a JS function that runs as-soon-as its defined.

Example usage :-

```
async function getdata () {
  return new Promise ((resolve, reject) => {
      // code
  })
}

IIFE
(async function main () {
    let a = await getdata()
    console.log (a)
}) ()
```

Destructuring

→ in Arrays.

It is an assignment used to unpack values from an array, or properties from objects, into distinct variables.

let [x, y] = [5, 7]

Here, 5 is assigned to x & 7 is assigned to y.

let [x, y] = [5, 7, 20]

Here, x & same as b4, and 20 is not assigned to anything.

let [x, y, ...rest] = [5, 7, 20, 24, 21]

Here ... → syntax, rest → Identifier.

x = 5, y = 7 & rest = [20, 24, 21]

→ in objects

spread operator

```
const obj = {
    a: 1,
    b: 2,
    c: 3
}

const {a, b} = obj
```

Spread Syntax.

it allows an iterable such as an array or ~~string~~ string to be expanded in places where 0 or more args. are expected. In an object literal the spread syntax enumerates the properties of an obj. and adds the key-value pairs to the object being created.

Example:-

```
const arr = [1, 7, 11]
const obj = {...arr}; // {0:1, 1:7, 2:11}
const add = sum(...arr) // 19
```

Hoisting

it refers to the process whereby the interpreter appears to move the declarations to the top of the code b4 execution.

Variables can thus be referenced b4 they are declared in JS.

JS only hoists declarations, not initialization. The variable will be undefined until the line where its initialized is reached.

Works only with var & function declarations and not let & const.